

Цикл for

Одно из преимуществ компьютеров перед людьми - способность повторять одни и те же действия многократно, быстро и совсем не утомляясь ☐

В Python существует две основных разновидности циклов:

- циклы, повторяющиеся определенное количество раз (for, счетные циклы, **counting loops**);
- циклы, повторяющиеся до наступления определенного события (while, условные циклы, **conditional loops**).

Цикл `for` замечательно работает, если мы заранее знаем, сколько повторений (итераций) нам требуется сделать.

Рассмотрим код, который распечатает 10 раз слово "Привет":

```
for i in range(10):  
    print('Привет')
```

Структура цикла `for` в Python выглядит так:

```
for название_переменной_цикла in range(количество_повторений):  
    блок кода
```

Двоеточие (:) в конце строки с инструкцией `for` сообщает интерпретатору Python, что дальше находится **блок команд**. В блок команд входят все строки, расположенные с отступом от строки с инструкцией `for`, вплоть до следующей строки без отступа.

Блок команд, который выполняется в цикле `for` называется **телом цикла**.

В предыдущих уроках мы считывали несколько чисел, при помощи нескольких команд `input`. С помощью цикла `for` можно считывать и обрабатывать сколько угодно чисел.

Рассмотрим следующий программный код:

```
for i in range(5):  
    num = int(input())  
  
    print('Квадрат вашего числа равен:', num * num)  
  
print('Цикл завершен')
```

Такая программа считывает 5 чисел и выводит на экран их квадраты вместе с поясняющей надписью. Поскольку вторая и третья строки выделены отступом, Python считает, что это тело цикла, которое выполняется 5 раз. Четвертая строка

не содержит отступа, поэтому не является частью цикла и будет выполнена всего один раз, после того как цикл завершится.

Примеры использования цикла for

Рассмотрим следующий программный код:

```
print('A')  
  
print('B')  
  
for i in range(5):  
  
    print('C')  
  
    print('D')  
  
print('E')
```

Результатом выполнения такой программы будут строки

A

B

C

D

C

D

C

D

C

D

E

То есть сначала программа распечатает символы `A` и `B`, затем символы `C` и `D` пять раз, а затем распечатает символ `E` один раз. Тело цикла состоит из двух строк: четвертой и пятой и именно они будут повторяться.

В программе может быть сколько угодно циклов. Например, если мы хотим, чтобы сначала 5 раз был распечатан символ `C`, а затем 5 раз символ `D`, мы можем использовать 2 цикла:

```
print('A')

print('B')

for i in range(5):

    print('C')

for i in range(5):

    print('D')

print('E')
```

Результатом выполнения такой программы будут строки:

A
B
C
C
C
C
C
C
D
D
D
D
D
E

Примечания

Примечание 1. Однократное выполнение тела цикла называется **итерацией цикла**.

Примечание 2. Графическое представление цикла `for` имеет вид:

```
for название переменной in range (количество повторений):  
    блок кода
```

Примечание 3. Напомним, что **блоком кода** называют объединённые друг с другом строки. Они всегда связаны с определённой частью программы (например, с инструкцией `if` или `for`). В Python блоки кода формируются при помощи **отступов**.

Примечание 4. Слово `for` пишется маленькими буквами, первая строка должна заканчиваться двоеточием, и тело цикла должно быть выделено отступом.

Переменная цикла

Давайте еще раз взглянем на базовую структуру цикла `for`:

```
for название_переменной_цикла in range(количество_повторений):  
    блок кода
```

Не совсем понятно, для чего нужна и как работает переменная цикла.

Рассмотрим следующий код:

```
for i in range(10):  
    print(i)
```

Результатом выполнения такого кода будет:

```
0  
1  
2  
3  
4  
5  
6  
7
```

8

9

Когда цикл впервые начинает работу, Python устанавливает значение переменной цикла `i = 0`. Каждый раз, когда мы повторяем тело цикла, Python увеличивает значение переменной на 1.

Почему большинство программистов начинают цикл с 0, а не с 1? Раньше некоторые начинали цикл с 1, а некоторые с 0. Те и другие приводили весьма изощренные аргументы, споря о том, какой способ лучше. Но в конце концов победили сторонники второго варианта. С тех пор большинство начинает циклы с 0. В частности, в Python цикл `for` начинается с 0, однако в будущих уроках вы узнаете, как это изменить.

Поскольку переменная цикла `i` увеличивается на 1 каждый раз, то ее можно использовать для отслеживания номера итерации, на которой мы находимся в циклическом процессе.

Рассмотрим следующий код:

```
for i in range(10):  
    print(i, '-- Привет')
```

Результатом выполнения такого кода будет:

```
0 -- Привет  
1 -- Привет  
2 -- Привет  
3 -- Привет  
4 -- Привет  
5 -- Привет  
6 -- Привет  
7 -- Привет  
8 -- Привет  
9 -- Привет
```

Если мы хотим начать с 1, то можем написать код:

```
for i in range(10):  
    print(i + 1, '-- Привет')
```

Результатом выполнения такого кода будет:

```
1 -- Привет
2 -- Привет
3 -- Привет
4 -- Привет
5 -- Привет
6 -- Привет
7 -- Привет
8 -- Привет
9 -- Привет
10 -- Привет
```

Обратите внимание, за счет выражения `i + 1`, мы начинаем вывод с `1`, а не с `0`.

Имена переменных цикла

Ранее говорилось, что имена переменных должны носить осмысленный характер и описывать их назначение. Однако для переменных цикла иногда делается исключение. В программировании для переменных цикла **обычно** используют буквы `i, j, k`.

Следующие две программы абсолютно одинаковые: в первой программе переменная цикла имеет название `i`, во второй программе - `number`:

```
for i in range(10):
    print(i)

for number in range(10):
    print(number)
```

Результатом выполнения обеих программ будет:

```
0
1
2
3
```

4

5

6

7

8

9

Почему для переменной циклов зарезервированы буквы `i, j, k`? Дело в том, что раньше программы использовались для математических расчетов, а в математике буквы `a, b, c` и `x, y, z` уже зарезервированы для других целей. Поэтому программисты выбрали для этой цели переменные `i, j, k`, и это стало общепринятой практикой.

Бывают ситуации, когда переменная цикла не используется в теле цикла. В таком случае, вместо того, чтобы давать ей имя, мы можем указать символ нижнего подчеркивания `_`:

```
for _ in range(5):  
    print('Python - awesome!')
```

Результатом выполнения такого кода будет:

```
Python - awesome!
```

```
Python - awesome!
```

```
Python - awesome!
```

```
Python - awesome!
```

```
Python - awesome!
```

Если переменная цикла не используется в теле цикла, то указывайте вместо нее символ нижнего подчеркивания `_`.

Примечания

Примечание. Следует помнить, что правая граница цикла в Python всегда не включительна. Таким образом следующий код:

```
for i in range(5):  
    print(i)
```

распечатает числа от 0 до 4:

```
0
1
2
3
4
```

Если требуется распечатать числа от 1 до 5, то мы пишем код:

```
for i in range(5):
    print(i + 1)
```

Функция range() с одним параметром

Рассмотрим программный код:

```
for i in range(10):
    print('Привет', i)
```

Значение, которое мы указываем в скобках у функции `range()` обозначает количество итераций цикла, при этом переменная `i` принимает последовательно значения: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

Если быть более точным, то мы говорим, что функция `range(n)` **генерирует последовательность** чисел от 0 до `n-1`, а цикл `for` последовательно перебирает эту последовательность.

Перегрузка range() с двумя параметрами

Если мы хотим начинать последовательность не с 0, а с какого-то другого числа, то мы можем использовать **перегрузку функции** `range()` принимающую два параметра. Например, вызов функции `range(1, 5)` сгенерирует последовательность чисел 1, 2, 3, 4 (будьте внимательны, правая граница не включительна). Если нам нужны числа от 1 до 5 включительно, то мы используем `range(1, 6)`.

Таким образом:

- `range(n)` : создает последовательность чисел 0, 1, 2, 3, ..., `n - 1`;
- `range(n, m)` : создает последовательность чисел `n`, `n + 1`, `n + 2`, ..., `m - 1`.

Напишем программу, которая выводит те числа из промежутка [100;999][100;999], которые оканчиваются на 7.

Используя функцию `range()` с двумя параметрами, получаем:

```
for i in range(100, 1000): # перебираем числа от 100 до 999

    if i % 10 == 7:        # используем остаток от деления на 10, для
        # получения последней цифры

        print(i)
```

Обратите внимание, в качестве второго параметра мы передали число 1000.

Если первый параметр больше второго, то функция `range()` генерирует пустую последовательность. Например, вызов функции `range(10, 1)` приводит к генерации пустой последовательности.

Перегрузка range() с 3 параметрами

Передавая два параметра в функцию `range()` мы можем генерировать любую последовательность целых чисел с шагом 1. Но, что делать если нужно поменять шаг? Как быть если мы хотим сгенерировать последовательность чисел 5, 10, 15, 20, 25? В этом случае существует еще одна перегрузка функции `range()`, принимающая три параметра: `range(n, m, k)`. Первый параметр задает **старт последовательности**, второй параметр задает **стоп последовательности** и третий – **шаг генерации** чисел.

Например, вызов функции `range(1, 10, 2)` создаст последовательность чисел 1, 3, 5, 7, 9, а вызов функции `range(5, 30, 5)` сгенерирует последовательность 5, 10, 15, 20, 25.

Напишем программу, которая выводит все четные числа из промежутка [56;170][56;170].

Используя функцию `range()` с тремя параметрами, получаем:

```
for i in range(56, 171, 2):

    print(i)
```

Обратите внимание, мы можем использовать функцию `range()` с двумя параметрами:

```
for i in range(56, 171):

    if i % 2 == 0:

        print(i)
```

однако такой код получается менее эффективным.

Отрицательный шаг генерации

Если шаг генерации является положительным числом, то генерируемая последовательность будет **возрастать**. Мы можем указать отрицательный шаг генерации (третий параметр), что приведет к генерированию **убывающей** последовательности.

В случае отрицательного шага, мы должны гарантировать, что старт последовательности (первый параметр) больше чем конец последовательности (второй параметр).

Например, вызов функции `range(20, 16, -1)` создаст последовательность чисел `20, 19, 18, 17`, а вызов функции `range(20, 10, -3)` сгенерирует последовательность `20, 17, 14, 11`.

Напишем программу, которая отсчитывает от 5 до 1, а затем выводит текст `Взлетаем!!!`:

```
for i in range(5, 0, -1):  
  
    print(i, end=' ')  
  
print('Взлетаем!!!')
```

Результатом будет:

```
5 4 3 2 1 Взлетаем!!!
```

Если величина шага отрицательна и первый параметр меньше второго, то функция `range()` генерирует пустую последовательность. Например, вызов функции `range(1, 10, -1)` приводит к генерации пустой последовательности.

Примеры использования функции range()

Вызов функции	Последовательность чисел
<code>range(10)</code>	<code>0, 1, 2, 3, 4, 5, 6, 7, 8, 9</code>
<code>range(1, 10)</code>	<code>1, 2, 3, 4, 5, 6, 7, 8, 9</code>
<code>range(3, 7)</code>	<code>3, 4, 5, 6</code>
<code>range(7, 3)</code>	пустая последовательность
<code>range(2, 15, 3)</code>	<code>2, 5, 8, 11, 14</code>

Вызов функции	Последовательность чисел
<code>range(9, 2, -1)</code>	9, 8, 7, 6, 5, 4, 3
<code>range(3, 10, -2)</code>	пустая последовательность

Примечания

Примечание 1. Функция `range()` может принимать от одного до трех параметров: `range(n)`, `range(n, m)`, `range(n, m, k)`

- первый параметр – это старт последовательности (включительно);
- второй параметр – это стоп последовательности (не включительно);
- третий параметр – это величина шага.

Примечание 2. Функция `range()` может генерировать только целые числа, включая отрицательные.

Примечание 3. Величина шага не может равняться нулю. Следующий код:

```
for i in range(1, 10, 0):
    print(i)
```

приведет к ошибке **ValueError: range() arg 3 must not be zero.**